

Security and Privacy in Federated Learning

Jonas Müller, Rebekka Schnoor

21.05.2026

Agenda

Secure Aggregation

Motivation

Even if the datasets stay with the clients, shared gradients can disclose information and enable inference attacks for both centralized and distributed FL.

Motivation

Even if the datasets stay with the clients, shared gradients can disclose information and enable inference attacks for both centralized and distributed FL.

Threat Models

- Honest-but-curious

The participant follows the protocol honestly but tries to learn as much as possible from the received messages

Motivation

Even if the datasets stay with the clients, shared gradients can disclose information and enable inference attacks for both centralized and distributed FL.

Threat Models

- **Honest-but-curious**

The participant follows the protocol honestly but tries to learn as much as possible from the received messages

- **Malicious**

The participant can lie or break the protocol

Motivation

Even if the datasets stay with the clients, shared gradients can disclose information and enable inference attacks for both centralized and distributed FL.

Threat Models

- **Honest-but-curious**

The participant follows the protocol honestly but tries to learn as much as possible from the received messages

- **Malicious**

The participant can lie or break the protocol

- **Malicious + (limited) access to memory**

The malicious participant additionally has access to some of the other participant's local memory (i.e. learns their secrets)

Secure Aggregation

Motivation

Even if the datasets stay with the clients, shared gradients can disclose information and enable inference attacks for both centralized and distributed FL.

Threat Models

- **Honest-but-curious**
Follow protocol but maximize learning
- **Malicious**
Can break the protocol
- **Malicious + (limited) access to memory**
+ Partial access to secrets

Challenges

- **User dropout**
- **Limited bandwidth and/or computation power**
- **Transmission of high dimensional information vectors (gradients)**
- **Unauthenticated network model**

Secure Aggregation Protocol by Bonawitz et al. [1]

Protocol Summary

- Uses masking vectors to encrypt individual gradients that cancel out in the aggregation
- Robust to user dropout by using secret sharing and a second masking vector
- Secure against malicious adversaries with limited memory access by using OneTime Pads and a sufficiently high threshold for secret reconstruction

Costs

for n users and vector dimension k

computation

User $O(n^2 + kn)$

Server $O(kn^2)$

communication

User $O(n + k)$

Server $O(n^2 + kn)$

storage

User $O(n + k)$

Server $O(n^2 + k)$

2 Main Variants:

- Fully Homomorphic Encryption (FHE)

Uses additively homomorphic encryption scheme

Encryption key is split among clients using secret sharing

1. Clients send encrypted gradients to the server
2. Server aggregates encrypted gradients and sends result to clients
3. Clients can then jointly decrypt the result

- Secure Multi-Party Computation (MPC) / Secure Two Party Computation (2PC)

1. Clients split their updates among N non-colluding servers using secret sharing
2. Servers jointly aggregate and send shares back to clients
3. Clients reconstruct the result

References

- [1] Keith Bonawitz u. a. *Practical Secure Aggregation for Federated Learning on User-Held Data*. 2016. arXiv: 1611.04482 [cs.CR]. URL: <https://arxiv.org/abs/1611.04482>.
- [2] Hossein Fereidooni u. a. *SAFELearn: Secure Aggregation for private FEderated Learning*. Cryptology ePrint Archive, Paper 2021/386. 2021. URL: <https://eprint.iacr.org/2021/386>.

Secure Aggregation Protocol by Bonawitz et al. [1]

1. Users pairwise agree on input perturbation pair $p_{u,v}, p_{v,u}$
2. Each user masks their gradient using the sum of all perturbations known and sends the result to the Server
3. The Server aggregates as usual because the perturbations are structured to cancel each other

Secure Aggregation Protocol by Bonawitz et al. [1]

1. Users pairwise agree on input perturbation pair $p_{u,v}, p_{v,u}$
 2. Each user masks their gradient using the sum of all perturbations known and sends the result to the Server
 3. The Server aggregates as usual because the perturbations are structured to cancel each other
- + information theoretically secure through OneTime Pads, even for malicious adversaries with limited memory access

Secure Aggregation Protocol by Bonawitz et al. [1]

1. Users pairwise agree on input perturbation pair $p_{u,v}, p_{v,u}$
 2. Each user masks their gradient using the sum of all perturbations known and sends the result to the Server
 3. The Server aggregates as usual because the perturbations are structured to cancel each other
- + information theoretically secure through OneTime Pads, even for malicious adversaries with limited memory access
 - ! strong assumptions:
 - pairwise secure channels with sufficient bandwidth for secret exchange
 - protocol completion by all users

Secure Aggregation Protocol by Bonawitz et al. [1]

0. Each user creates a key pair and broadcasts their public key to all other users
1. Perturbation vector agreement
 - 1.1 Secret sharing: Users distribute perturbation vector among all other users with reconstruction threshold t
2. Masking + sending of gradients
 - 2.1 Only use perturbations shared between surviving users. Reveal secrets of dropped out users if at least t users survived
3. Aggregation by server

Secure Aggregation Protocol by Bonawitz et al. [1]

- 0. Each user creates a key pair and broadcasts their public key to all other users + Robustness to user dropout
- 1. Perturbation vector agreement
 - 1.1 Secret sharing: Users distribute perturbation vector among all other users with reconstruction threshold t
- 2. Masking + sending of gradients
 - 2.1 Only use perturbations shared between surviving users. Reveal secrets of dropped out users if at least t users survived
- 3. Aggregation by server

Secure Aggregation Protocol by Bonawitz et al. [1]

- 0. Each user creates a key pair and broadcasts their public key to all other users
 - 1. Perturbation vector agreement
 - 1.1 Secret sharing: Users distribute perturbation vector among all other users with reconstruction threshold t
 - 2. Masking + sending of gradients
 - 2.1 Only use perturbations shared between surviving users. Reveal secrets of dropped out users if at least t users survived
 - 3. Aggregation by server
- + Robustness to user dropout
 - ! Secret exposure in case of malicious server or delayed update sending; increased communication cost

Secure Aggregation Protocol by Bonawitz et al. [1]

0. Key pair generation + generation of masking vector b
1. Perturbation vector agreement
 - 1.1 Secret sharing including b
2. Masking + sending of gradients: Use b as second mask
 - 2.1 Reveal secrets of dropped out users if at least t users survived: Each user will only reveal a share of p_u OR b_u
3. Aggregation by server

Secure Aggregation Protocol by Bonawitz et al. [1]

0. Key pair generation + generation of masking vector b
 1. Perturbation vector agreement
 - 1.1 Secret sharing including b
 2. Masking + sending of gradients: Use b as second mask
 - 2.1 Reveal secrets of dropped out users if at least t users survived: Each user will only reveal a share of p_u OR b_u
 3. Aggregation by server
- + For $t > \frac{n}{2}$, all gradients stay masked by either p or b ; extend to stronger adversaries by increasing threshold

Secure Aggregation Protocol by Bonawitz et al. [1]

0. Key pair generation + generation of masking vector b
 1. Perturbation vector agreement
 - 1.1 Secret sharing including b
 2. Masking + sending of gradients: Use b as second mask
 - 2.1 Reveal secrets of dropped out users if at least t users survived: Each user will only reveal a share of p_u OR b_u
 3. Aggregation by server
- + For $t > \frac{n}{2}$, all gradients stay masked by either p or b ; extend to stronger adversaries by increasing threshold
 - ! Communication cost of $O(kn^2)$

Secure Aggregation Protocol by Bonawitz et al. [1]

0. Key pair generation + generation of masking vector b
 1. Perturbation vector agreement
 - 1.1 Secret sharing including b
 2. Masking + sending of gradients: Use b as second mask
 - 2.1 Reveal secrets of dropped out users if at least t users survived: Each user will only reveal a share of p_u OR b_u
 3. Aggregation by server
- + For $t > \frac{n}{2}$, all gradients stay masked by either p or b ; extend to stronger adversaries by increasing threshold
 - ! Communication cost of $O(kn^2)$

Reducing Complexity

Share seeds for PRG instead of full vectors using DH Key Exchange via server

Secure Aggregation Protocol by Bonawitz et al. [1]

0. Key pair generation + generation of masking vector b
 1. Perturbation vector agreement
 - 1.1 Secret sharing including b
 2. Masking + sending of gradients: Use b as second mask
 - 2.1 Reveal secrets of dropped out users if at least t users survived: Each user will only reveal a share of p_u OR b_u
 3. Aggregation by server
- + For $t > \frac{n}{2}$, all gradients stay masked by either p or b ; extend to stronger adversaries by increasing threshold
- ! Communication cost of $O(kn^2)$

Reducing Complexity

Share seeds for PRG instead of full vectors using DH Key Exchange via server

Getting rid of secured channel assumption

Setup secured channels via server-mediated DH $\rightarrow$ adds PFS